

LAPORAN ANALISIS

Laporan Analisis Coding Style Project

Analisis struktur, arsitektur, konvensi kode, komunikasi frontend-backend, dan pola implementasi.

Project: test-app

Stack: Laravel 12, PHP 8.2+, Vue 3, Vite, Axios, Tailwind CSS

Tanggal analisis: 23 Juli 2026

Ruang lingkup: backend, frontend, database, routes, konfigurasi, dan test yang tersedia.

1. Ringkasan Eksekutif

Project ini adalah aplikasi manajemen toko berbasis:

- **Backend:** Laravel 12, PHP 8.2+

- **Frontend:** Vue 3, Vue Router, Vite, Tailwind CSS
- **HTTP client:** Axios
- **Authentication:** Laravel Sanctum dengan Bearer Token
- **Database:** Mendukung konfigurasi SQLite/MySQL/MariaDB/PostgreSQL/SQL Server, tetapi implementasi bisnis tertentu bergantung pada MySQL
- **UI utility:** SweetAlert2, toast custom, Lucide Vue
- **Real-time:** Laravel Broadcasting melalui `window.Echo`

Arsitekturnya menggunakan pendekatan **modular berdasarkan domain bisnis**, tetapi belum sepenuhnya menerapkan pola clean architecture atau domain-driven design. Pola yang paling dominan adalah:

```
Vue View
  -> Composable
  -> Module Service
    -> Axios API Client
    -> Laravel Route
    -> Controller
    -> Service
      -> Eloquent Model / Query Builder / Stored Procedure
```

Tidak ditemukan Repository Pattern, Form Request khusus, Pinia, Vuex, atau API Resource.

2. Struktur Folder Project

Struktur utama:

```
app/
├── Events/
├── Http/
│   ├── Controllers/
│   └── Middleware/
├── Models/
├── Providers/
└── Services/

database/
├── factories/
├── migrations/
└── seeders/

resources/
├── css/
├── js/
│   ├── assets/
│   ├── layouts/
│   ├── modules/
│   └── utilities/
└── views/

routes/
├── api.php
├── console.php
└── web.php

tests/
```

```
| Feature/  
| Unit/
```

Frontend dikelompokkan berdasarkan modul bisnis, misalnya:

```
resources/js/modules/jabatan/  
| views/  
| components/  
| services/  
| composables/
```

Pola ini juga diterapkan pada modul:

- produk
- karat
- jenisproduk
- pelanggan
- nampan
- saldo
- transaksi
- offtake
- pembeliandaritoko
- authentication
- dan modul lainnya

Struktur folder frontend relatif baik untuk aplikasi besar karena setiap fitur memiliki batas folder sendiri. Entry point frontend berada pada `resources/js/app.js`, sedangkan fallback SPA berada pada `routes/web.php`.

3. Arsitektur Aplikasi

Backend

Backend menggunakan pola:

```
Route -> Controller -> Service -> Model/Database
```

Controller menerima HTTP request, melakukan validasi, memanggil service, lalu membentuk JSON response.

Service bertanggung jawab terhadap:

- Query data
- Create/update/delete
- Business rules
- Database transaction
- Perhitungan transaksi
- Integrasi Telegram
- Pengelolaan status produk dan saldo

Contoh dependency injection service terlihat pada `app/Http/Controllers/Master/JabatanController.php`. Contoh kompleksitas business logic berada pada `app/Services/Transaksi/TransaksiService.php`.

Frontend

Frontend menggunakan pendekatan SPA:

```
App.vue
  -> Vue Router
    -> Layout
      -> Module View
        -> Module Components
          -> Module Composable
            -> Module Service
              -> apiClient Axios
```

Routing dan permission dikontrol pada `resources/js/router/index.js`.

Kesimpulan Arsitektur

Arsitektur saat ini dapat dikategorikan sebagai:

Modular monolith dengan service layer dan Vue feature modules.

Kelebihan:

- Pemisahan domain cukup jelas.
- Controller tidak seluruhnya menampung business logic.
- Frontend memiliki pembagian `views`, `components`, `services`, dan `composables`.
- Authentication dan authorization sudah dipisahkan secara konseptual.

Keterbatasan:

- Service transaction sangat besar dan menangani terlalu banyak tanggung jawab.
- Tidak ada abstraction layer repository.
- Format API belum distandarkan secara global.
- Error handling masih dilakukan berulang di banyak controller.
- Beberapa business rule masih langsung menggunakan `DB::table()` dari service.

4. Pola Komunikasi Frontend dan Backend

Komunikasi frontend-backend menggunakan Axios melalui satu client bersama:

`resources/js/utilities/api/apiClient.js`

Konfigurasi utama:

- `baseUrl` berasal dari `VITE_API_URL`.
- Header `Accept: application/json`.
- Header `Content-Type: application/json`.
- Bearer token dibaca dari `localStorage`.
- `FormData` menghapus `Content-Type` agar browser menentukan boundary multipart.
- Response interceptor menangani status `401`.
- Jika token tidak valid, frontend membersihkan local storage dan mengarahkan user ke `/login`.

Setiap modul memiliki service API sendiri, contohnya:

resources/js/modules/jabatan/services/jabatanService.js

Pola pemanggilan umumnya:

```
api.get(...)  
api.post(...)  
return response.data
```

Sebagian besar service memiliki pola:

```
try {  
  const response = await api.get(...);  
  return response.data;  
} catch (error) {  
  throw error;  
}
```

try/catch tersebut umumnya tidak menambah behavior baru sehingga berfungsi sebagai pass-through saja.

5. Style Penamaan File

Backend PHP

- Controller menggunakan PascalCase: `JabatanController.php`, `TransaksiController.php`, `AuthenticationController.php`.
- Service menggunakan PascalCase: `JabatanService.php`, `ProdukService.php`.
- Model menggunakan PascalCase.
- Folder domain menggunakan PascalCase: `Master`, `Produk`, `Transaksi`, `Pelanggan`.
- Migration menggunakan timestamp Laravel dan snake_case, seperti `create_jabatan_table`, `create_transaksi_table`, dan `create_cetak_nota_pembelian_procedure`.

Frontend JavaScript

- Service file menggunakan camelCase lowercase: `jabatanService.js`, `authenticationService.js`, `pembeliandaritokoService.js`.
- Composable file menggunakan format `useJabatan.js`, `useAuthentication.js`, dan `usePembelianDariToko.js`.
- Component Vue menggunakan PascalCase: `JabatanTable.vue`, `JabatanModal.vue`, dan `PointOfSaleView.vue`.
- View juga menggunakan PascalCase.

Ketidakkonsistenan

- Suplier digunakan secara konsisten, tetapi secara bahasa Inggris biasanya `Supplier`.
- `jenisproduk`, `jeniskarat`, `mutasisaldo`, dan `pembeliandaritoko` menggunakan lowercase tanpa separator.
- `CetakNotaPenjualan` pada route dan method menggunakan PascalCase, berbeda dari mayoritas method PHP yang camelCase.
- `sendnotification` pada route menggunakan lowercase tanpa separator.
- Beberapa variabel frontend memakai PascalCase walaupun bukan class, misalnya `PelangganList`, `DiskonList`, `TransaksiID`, dan `PembelianDariToko`.

6. Style Penamaan Class

Class PHP konsisten memakai PascalCase:

```
JabatanController  
JabatanService  
Jabatan  
TransaksiUpdated  
CheckPermission
```

Class Vue tidak memakai class JavaScript karena seluruh komponen menggunakan `<script setup>`. Nama file component menjadi identitas component.

Class names pada backend cukup jelas dan mengikuti domain bisnis. Namun beberapa class memiliki tanggung jawab terlalu besar, terutama service transaksi.

7. Style Penamaan Function

Backend

Method menggunakan camelCase dan umumnya memakai prefix:

- `get...`
- `store...`
- `update...`
- `delete...`
- `create...`
- `handle...`
- `process...`

Contoh: `getJabatan`, `storeJabatan`, `updateJabatan`, `deleteJabatan`, `generateKodeTransaksi`, dan `processPayment`.

Pola CRUD cukup konsisten, tetapi terdapat variasi:

- Controller memakai `storeJabatan`, service memakai `createJabatan`.
- `batalDetail`, `batalTransaksiDetail`, `cancelTransaksi` mencampur bahasa Indonesia dan Inggris.
- `CetakNotaPenjualan` menggunakan PascalCase.
- Beberapa nama method sangat panjang, misalnya `storeProdukToPembelianDetailDariLuar`.

Frontend

Composable menggunakan function expression:

```
const fetchJabatan = async () => {}  
const submitJabatan = async () => {}  
const handleDelete = async () => {}
```

Pola function yang berulang adalah `fetchX`, `handleCreate`, `handleEdit`, `submitX`, `handleDelete`, dan `handleRefresh`. Untuk modul transaksi, function lebih spesifik terhadap proses bisnis seperti `handlePayment`, `handleNextOrder`, `fetchKodeTransaksi`, dan `handlePilihProduk`.

8. Style Penamaan Variable

PHP

Variable menggunakan camelCase: `$jabatanService`, `$validatedData`, `$imageFile`, `$saldoAktif`, `$transaksi`, dan `$produk`.

Namun ada variasi kapitalisasi domain seperti `$mutasisaldoService`, `$jenisProdukService`, dan `$namaKolomPoin`.

JavaScript

Umumnya menggunakan camelCase: `isLoading`, `searchQuery`, `currentPage`, `itemsPerPage`, `isModalOpen`, dan `selectedProdukIds`.

State boolean konsisten memakai prefix `is`: `isLoading`, `isEdit`, `isSubmitting`, `isModalOpen`, dan `isAccessDeniedOpen`.

Pengecualian yang kurang konsisten adalah `PelangganList`, `DiskonList`, `TransaksiID`, `TransaksiDetail`, dan `PembelianDariToko`.

9. Pola Validasi

Backend

Validasi mayoritas dilakukan langsung di controller menggunakan `$request->validate([...])`. Sebagian besar validasi berbasis string rule, diletakkan di masing-masing method controller, dan tidak menggunakan Form Request.

Terdapat pengecualian di `UserController` yang memakai `Validator::make(...)`. Validasi authentication juga dilakukan langsung di controller pada `AuthenticationController.php`.

Frontend

Frontend melakukan validasi manual di composable. Field kosong diperiksa dengan `if (!formJabatan.jabatan)`, error disimpan pada `errors.value`, dan error backend HTTP 422 disalin ke state frontend.

Validasi login menggunakan regex email, validasi password kosong, dan validasi panjang password minimal.

Evaluasi

Validasi dilakukan di dua tempat:

```
Frontend validation -> Backend validation
```

Ini baik untuk pengalaman pengguna dan keamanan, tetapi aturan validasi berpotensi tidak sinkron karena tidak memiliki shared schema atau definisi bersama.

10. Pola Response API

Response API dibuat manual menggunakan `response()->json()`. Pola umum CRUD master:

```
{
  "status": 200,
  "success": true,
```

```
"message": "Data berhasil ditemukan",  
"data": []  
}
```

Response login memiliki pola berbeda dengan `access_token`, `token_type`, dan `user`. Response transaksi lebih bervariasi dan kadang menggunakan `success`, `status`, `message`, `data`, dan `errors`.

Ketidakkonsistenan utama

- Tidak semua response memiliki field `success`.
- Tidak semua response memiliki field `data`.
- Error validasi authentication memakai HTTP 421, bukan HTTP 422.
- Sebagian kondisi gagal memakai HTTP 200.
- Sebagian response menggunakan `status: false`, bukan `success: false`.
- Response permission hanya berisi `message`.
- Belum ada API Resource atau response formatter global.

11. Error Handling

Backend

Error handling menggunakan kombinasi `try/catch` di controller, `try/catch` di service, `\Exception`, `ValidationException`, `firstOrFail()`, dan response JSON manual.

Controller transaksi sering mengembalikan `'message' => 'Gagal: ' . $e->getMessage()`. Beberapa exception message internal diteruskan langsung ke client. Ini memudahkan debugging, tetapi berisiko mengekspos detail internal pada production.

Frontend

Error handling berada di composable dengan pola `error.response?.data?.message`, `error.response?.data?.errors`, dan `error.response?.status`.

Feedback user menggunakan toast untuk error umum, SweetAlert untuk konfirmasi delete, modal khusus access denied, dan redirect otomatis ke login pada status 401.

Error handling global

Error handling global terlihat jelas untuk status 401 di Axios interceptor. Error lain masih ditangani per composable. Belum terlihat formatter error global atau centralized notification handler.

12. Pola Database

Database menggunakan Eloquent Model, Query Builder melalui `DB::table()`, `DB::transaction()`, foreign key, status integer sebagai penanda aktif/nonaktif, timestamp Laravel, dan stored procedure untuk pencetakan nota.

Status dan soft delete

Project tidak terlihat menggunakan trait `SoftDeletes`, tetapi banyak model memiliki `protected $hidden = ['created_at', 'updated_at', 'deleted_at'];`. Migration yang dibaca tidak selalu memiliki kolom `deleted_at`.

Artinya, `status` integer lebih banyak digunakan sebagai soft-state manual: 1 aktif, 0 tidak aktif, dan 2 sudah diproses/lunas/keluar, tergantung domain. Makna status belum distandarkan sebagai enum atau constant.

Foreign key

Foreign key cukup banyak digunakan dengan `onDelete('cascade')`. Terdapat penggunaan relasi berbasis business key, misalnya relasi transaksi berdasarkan `kode`, bukan hanya ID.

Query style

Terdapat tiga gaya database: Eloquent, Query Builder, dan Stored Procedure melalui `DB::unprepared()`. Stored procedure seperti `create_cetak_nota_pembelian_procedure.php` mengindikasikan ketergantungan pada fitur database tertentu, terutama MySQL.

13. Pola Service, Repository, dan Controller

Controller

Controller biasanya menerima `Request`, validasi request, memanggil service, mengecek hasil service, dan membentuk JSON response. Controller sudah relatif tipis untuk CRUD sederhana.

Service

Service memiliki beberapa peran sekaligus: query data, data transformation, business validation, transaction management, authentication context melalui `Auth`, file upload, integrasi API eksternal, pengelolaan saldo dan poin, serta broadcasting-related flow.

Service transaksi jauh lebih besar dan mengandung banyak aturan bisnis dalam satu class.

Repository

Tidak ditemukan folder atau class Repository. Semua query langsung dilakukan melalui Model Eloquent, `DB::table()`, dan `DB::unprepared()`.

Dependency Injection

Dependency injection digunakan pada controller dan beberapa service. Namun beberapa service mengakses global facade/context seperti `Auth::id()`, `request()->user()`, `DB`, dan `Http`, sehingga dependency belum sepenuhnya eksplisit.

14. Pola Component Vue

Seluruh component Vue yang diperiksa menggunakan:

```
<script setup>
```

Tidak ditemukan `export default` atau `defineComponent` pada component utama.

Pola component:

```
View
├── Table
├── Modal
└── Form atau subcomponent khusus
```

Contoh:

```
jabatan/  
├─ views/JabatanView.vue  
├─ components/JabatanTable.vue  
├─ components/JabatanModal.vue  
├─ services/jabatanService.js  
└─ composables/useJabatan.js
```

View

View menjadi page entry, menggunakan `onMounted`, memanggil composable, serta menampilkan table dan modal.

Table

Table menangani search input, loading state, pagination, tombol edit, tombol delete, dan refresh.

Modal

Modal menangani form input, mode create/edit, error field, submit event, dan file upload pada modul tertentu.

Style component

Tailwind digunakan langsung pada template dengan class yang panjang. Design system belum terlihat memiliki komponen UI abstraction bersama untuk Button, Input, Modal, Table, Pagination, dan Loading state.

15. Pola Penggunaan Axios

Semua module service mengimpor API client terpusat:

```
import api from '../../utilities/api/apiClient';
```

Method service mengembalikan `response.data`, sehingga composable bekerja langsung dengan payload API.

Pola endpoint memakai GET untuk mengambil data dan POST untuk create, update, serta delete. Operasi update/delete belum mengikuti pola HTTP REST konvensional `PUT/PATCH/DELETE`.

Axios interceptor menangani Authorization Bearer token, FormData, dan redirect untuk 401. Belum ada interceptor khusus untuk standarisasi error response, global loading indicator, retry, transformasi response, atau penanganan 403, 422, dan 500 secara terpusat.

16. Pola Composable

Composable menjadi pusat state management per feature.

```
const data = ref([]);  
const isLoading = ref(false);  
const searchQuery = ref('');  
const currentPage = ref(1);  
const itemsPerPage = ref(10);  
const isEdit = ref(false);
```

```
const isModalOpen = ref(false);  
const errors = ref({});
```

Composable menyediakan state, computed filtering, computed pagination, CRUD operation, modal state, form state, dan error state.

Contoh representatif terdapat pada `resources/js/modules/jabatan/composables/useJabatan.js`.

Shared state

Beberapa composable menyimpan `ref` pada module scope sebelum function composable dipanggil. Ini membuat state bersifat shared singleton antar-component yang memakai composable tersebut.

Authentication juga menggunakan module-scope state pada `useAuthentication.js`.

Kelebihan

- Sederhana.
- Mudah dipakai lintas component.
- Tidak membutuhkan library state management tambahan.

Risiko

- Lifecycle state lebih sulit diprediksi.
- State antar-instance composable bisa saling berbagi tanpa deklarasi eksplisit.
- Modul transaksi memiliki state yang sangat banyak dalam satu composable.
- Sebagian composable sudah cukup besar dan berpotensi menjadi god composable.

17. Pola Store

Tidak ditemukan penggunaan Pinia, Vuex, `defineStore`, `createPinia`, maupun `useStore`.

State global dibuat melalui module-scope `ref`, `localStorage`, `provide/inject` untuk toast, dan utility state seperti `themeState.js` serta `sidebarState.js`.

Authentication state disimpan di composable dan `localStorage`, bukan store resmi.

18. Pola Coding yang Berulang

Backend CRUD

Setiap domain mengikuti pola `getX`, `storeX`, `updateX`, dan `deleteX`.

Frontend CRUD

Hampir setiap module memiliki View, Table, Modal, Service, dan Composable. Composable mengulang fetch data, search, pagination, create/edit modal, submit, delete confirmation, toast, dan loading state.

Response API

Response manual menggunakan struktur berulang `status`, `success`, `message`, dan `data`, tetapi belum dipusatkan pada helper atau response class.

Error handling

Banyak method menggunakan pola:

```
try {
  ...
} catch (error) {
  const errorMessage = error.response?.data?.message || '...';
  toast.error(errorMessage);
}
```

Database transaction

Service transaksi menggunakan pola:

```
return DB::transaction(function () {
  ...
});
```

Ini tepat untuk operasi saldo, produk, detail transaksi, dan pembayaran, tetapi implementasinya masih tersebar di service yang besar.

Comment patch/debug

Beberapa file memiliki komentar seperti “KUNCI PERBAIKAN”, “PERUBAHAN DI SINI”, “Tambahkan...”, “Perbaiki...”, dan komentar beremoji. Ini menunjukkan sejarah debugging atau patch incremental masih tertinggal dalam source code.

19. Temuan Arsitektural Penting

1. **Tidak ada standard response global.** Frontend harus memahami beberapa bentuk payload yang berbeda.
2. **HTTP status belum konsisten.** Kondisi gagal kadang dikirim dengan HTTP 200.
3. **Validasi tersebar.** Tidak ada Form Request atau shared validation contract.
4. **Service transaksi terlalu besar.** Pembayaran, saldo, poin, produk, nampan, notifikasi Telegram, dan pembatalan transaksi berada pada service domain yang sama.
5. **Tidak ada Repository Pattern.** Query tersebar langsung pada service.
6. **Status integer memiliki banyak makna.** Nilai 0, 1, dan 2 perlu didefinisikan sebagai enum atau constant per domain.
7. **Ketergantungan database tidak seragam.** Testing memakai SQLite, sementara stored procedure membutuhkan MySQL.
8. **State management frontend informal.** Module-scope composable efektif untuk aplikasi sekarang, tetapi kurang eksplisit untuk skala yang lebih besar.
9. **Duplikasi frontend cukup tinggi.** Pola CRUD table/modal/composable berulang di banyak module.
10. **Error internal berpotensi diteruskan ke client.** Beberapa response memasukkan `$e->getMessage()` secara langsung.
11. **Coverage test sangat minim.** Test yang tersedia hanya test contoh Laravel pada `tests/Feature/ExampleTest.php` dan `tests/Unit/ExampleTest.php`.
12. **Naming belum sepenuhnya konsisten.** Ada campuran bahasa Indonesia-Inggris, camelCase-PascalCase untuk variable, serta beberapa typo domain seperti `Supplier`.

20. Kesimpulan

Coding style project ini dapat dirangkum sebagai:

Laravel modular monolith dengan Service Layer, Eloquent/Query Builder hybrid, dan Vue 3 feature-based frontend menggunakan composible sebagai state manager.

Style yang paling konsisten:

- Class PHP menggunakan PascalCase.
- Method PHP menggunakan camelCase.
- Component Vue menggunakan PascalCase.
- Composible menggunakan prefix `use`.
- State boolean menggunakan prefix `is`.
- Frontend API dipusatkan melalui Axios client.
- Business domain dipisahkan berdasarkan folder.

Area yang paling membutuhkan standardisasi secara arsitektural adalah:

- Kontrak response API.
- HTTP status code.
- Error handling.
- Form Request.
- Pembagian service transaksi.
- Definisi status database.
- State management frontend.
- Pengurangan duplikasi CRUD.
- Test untuk business flow transaksi dan authorization.

Secara keseluruhan, project sudah memiliki fondasi modular yang cukup baik, tetapi masih berada pada tahap **pragmatic application architecture**: efektif untuk pengembangan cepat, namun memerlukan standardisasi kontrak dan pemecahan tanggung jawab apabila project akan terus berkembang.

Tidak ada source code aplikasi yang diubah selama analisis dan pembuatan dokumen ini.